

Virtual Environment Setup

Create Virtual Environment

```
python -m venv venv
```

Activate Virtual Environment

```
.\venv\Scripts\Activate.ps1
```

Remove Virtual Environment (if needed)

```
Remove-Item -Recurse -Force .\venv
```

Required Libraries

```
pip install feature-engine  
pip install ucimlrepo # Required for sample data
```

Factor Analysis

Factor Analysis is a statistical technique used to identify underlying patterns in data by grouping correlated variables into factors. This project implements a comprehensive factor analysis pipeline with quality gates and detailed interpretations.

Pipeline Overview

The factor analysis pipeline consists of 6 steps:

1. **Data Preparation** - Data cleaning and preprocessing
2. **KMO Test** - Kaiser-Meyer-Olkin Test of Sampling Adequacy
3. **Bartlett's Test** - Bartlett's Test of Sphericity
4. **Eigenvalue Calculation** - Determining factor count and variance explained
5. **Factor Loadings** - Computing variable-factor associations
6. **Factor Grouping** - Creating interpretable factor groups

Step 1: Data Preparation ([prepare_factor_analysis_data](#))

Purpose

This function prepares raw data for factor analysis by performing essential preprocessing steps. It handles missing values, encodes categorical variables, and standardizes all features. The output is ready for correlation-based factor analysis.

Why Data Preparation Matters

Factor analysis relies on **correlation matrices**. Raw data often contains:

- Missing values
- Categorical (non-numeric) variables
- Variables on different scales (e.g., income vs. age)

Preprocessing ensures:

- No missing data interferes with correlations
- All variables are numeric
- Variables on similar scales so large-scale variables don't dominate

Processing Steps

```
Raw Data
  ↓
1. Check Data Quality
  - Remove zero-variance columns (no variation)
  - Remove columns with infinite values
  - Remove duplicate columns
  ↓
2. Handle Missing Values
  - Numeric columns: fill with mean/median/zero
  - Categorical columns: fill with most frequent category
  ↓
3. Encode Categorical Variables
  - One-hot encoding (default) or Ordinal encoding
  ↓
4. Standardize Features
  - Apply StandardScaler: (X - mean) / std
  ↓
Prepared Data (Standardized & Numeric)
```

Function Signature

```
prepare_factor_analysis_data(
    df: pd.DataFrame,
    target_variable: str | None = None,
    drop_last: bool = True,
    fill_strategy_numeric: str = "median",
    encoding_strategy_categorical: str = "ordinal",
```

```
output_dir: str | None = None
) -> (pd.DataFrame, dict)
```

Parameters

Parameter	Type	Default	Description
df	DataFrame	-	Raw input dataframe
target_variable	str or None	None	Name of target variable to exclude from analysis
drop_last	bool	True	Drop last category during one-hot encoding to avoid multicollinearity
fill_strategy_numeric	str	"median"	Strategy for missing numeric values: "mean", "median", or "zero"
encoding_strategy_categorical	str	"ordinal"	Encoding method: "onehot" or "ordinal"
output_dir	str or None	None	Directory to save data preparation metadata as JSON file

Example

Input Data:

	AGE	TENURE	INCOME	CITY	
0	25	1	30000	A	
1	30	3	40000	A	
2	35	5	50000	B	
3	NaN	2	35000	A	# Missing value

Output Data (Standardized):

	AGE	TENURE	INCOME	CITY_A
0	-1.2247	-1.1355	-1.0000	0.7071
1	0.0000	-0.1622	0.0000	0.7071
2	1.2247	1.2977	1.0000	-1.4142
3	-0.0000	-0.1622	-0.3333	0.7071

Data Quality Checks

The function performs several quality checks:

1. Data Type Validation

- Only numeric and object (categorical) data types are supported

- Raises error if datetime, boolean, or other types are present

2. Zero Variance Columns

- Identifies columns with only one unique value
- These add no information to correlations and are dropped

3. Infinite Values

- Detects and removes columns containing infinite values
- Prevents calculation errors in correlation matrix

4. Duplicate Columns

- Identifies columns with identical values
- Drops duplicates to avoid redundant analysis

5. Missing Values After Imputation

- Validates that all missing values are properly filled
- Raises error if any missing values remain

Imputation Strategies

Numeric Variables:

- "mean": Fill with column average (sensitive to outliers)
- "median": Fill with column median (robust to outliers) ✓ Recommended
- "zero": Fill with zero (use when zero is meaningful)

Categorical Variables:

- Always filled with the most frequent category (mode)

Encoding Strategies

One-Hot Encoding (`encoding_strategy_categorical="onehot"`):

- Creates binary (0/1) columns for each category
- `drop_last=True` drops the last category to avoid multicollinearity
- Example: CITY [A, B, C] → CITY_A, CITY_B (CITY_C dropped)

Ordinal Encoding (`encoding_strategy_categorical="ordinal"`):

- Assigns integer values to categories arbitrarily
- Example: CITY [A, B, C] → [1, 2, 3]
- More compact but assumes ordinality

Standardization (StandardScaler)

All variables are standardized to mean=0 and std=1:

$$Z = \frac{X - \mu}{\sigma}$$

Why Standardize?

- Factor analysis uses correlations, not covariances
- Standardization ensures all variables contribute equally
- Without it, high-scale variables (e.g., income in millions) would dominate

Returned Values

1. `df_prepared` (DataFrame)

- Cleaned, encoded, and standardized data
- Ready for KMO and Bartlett tests

2. `metadata` (Dict)

- Original column names
- Processed column names
- List of dropped columns (zero-variance, infinite, duplicates)
- Imputation and encoding strategies used

Usage Example

```
from factor_analysis import prepare_factor_analysis_data

# Prepare data for factor analysis
df_prepared, metadata = prepare_factor_analysis_data(
    df=df,
    target_variable="TARGET",
    drop_last=True,
    fill_strategy_numeric="median",
    encoding_strategy_categorical="ordinal"
)

# Access prepared data
print(df_prepared.head())
print(f"Shape: {df_prepared.shape}")

# Access metadata
print(f"Original columns: {metadata['originalColumns']}")
print(f"Dropped columns: {metadata['zeroVarianceColumnsDropped']}")
```

Notes

- Factor analysis is based on **correlations**, not raw values
- Standardization ensures all variables are on the same scale
- The target variable is temporarily removed during processing, then re-added at the end
- All logs are recorded for audit trail and debugging

Step 2: KMO Test (calculate_kmo_manual)

Purpose

The **Kaiser-Meyer-Olkin (KMO)** test measures whether the correlation structure of variables is suitable for factor analysis. It answers the question: *"Are the variables sufficiently correlated to justify grouping them into factors?"*

KMO is a **quality gate** — if $KMO < 0.50$, factor analysis is not recommended.

Why KMO Matters

Factor analysis works best when variables share **common variance**. KMO compares:

- **Common variance** (what variables share via correlations)
- **Unique variance** (what makes each variable special via partial correlations)

High KMO means variables share common factors. Low KMO means variables are too independent.

The KMO Formula

The KMO statistic compares squared correlations to squared partial correlations:

$$KMO = \frac{\sum(r_{ij}^2)}{\sum(r_{ij}^2) + \sum(p_{ij}^2)}$$

Where:

- r_{ij} = Correlation between variables i and j
- p_{ij} = Partial correlation between i and j (controlling for all other variables)

Key Insight: KMO prefers:

- ✓ High correlations (variables share common factors)
- ✓ Low partial correlations (no unique relationships after accounting for other variables)

Mathematical Intuition

Consider this example with 3 variables (X, Y, Z):

Matrix	X↔Y	X↔Z	Y↔Z
Squared Correlations	0.64	0.49	0.36
Squared Partial Corrs	0.04	0.01	0.09

Overall KMO:

$$KMO = \frac{0.64 + 0.49 + 0.36}{0.64 + 0.49 + 0.36 + 0.04 + 0.01 + 0.09} = \frac{1.49}{1.63} = 0.914$$

Interpretation: 0.914 is "Excellent" — variables share strong common variance.

Partial Correlation Explained

Partial Correlation measures the relationship between two variables after removing the effect of all other variables.

Formula for 3 variables (X, Y, Z):

$$p_{XY|Z} = \frac{r_{XY} - r_{XZ} \cdot r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}}$$

Why It Matters:

- High correlation but high partial correlation = relationship is not driven by common factors (bad for FA)
- High correlation but low partial correlation = relationship is driven by common factors (good for FA)

KMO Interpretation Scale

KMO Range	Interpretation	Suitability
< 0.50	Not suitable	Do not use factor analysis
0.50 - 0.60	Weak	Marginal, proceed with caution
0.60 - 0.70	Moderate	Acceptable
0.70 - 0.80	Good	Strong correlation structure
0.80 - 0.90	Very good	Very strong correlations
≥ 0.90	Excellent	Ideal for factor analysis

Variable-Level vs. Dataset-Level KMO

The function calculates **two types** of KMO:

1. KMO Per Variable (Variable-Level)

- Individual KMO for each variable
- Shows which variables are most suitable
- Formula: $KMO(i) = \frac{\sum(r_{ij}^2)}{\sum(r_{ij}^2) + \sum(p_{ij}^2)}$ (for variable i only)

2. KMO Model (Dataset-Level)

- Single aggregate KMO for all variables combined
- Used as quality gate for entire analysis
- Formula: Same as above, but sums across ALL variables

Function Signature

```
calculate_kmo_manual(
    df: pd.DataFrame,
    target_variable: str | None = None,
```

```
output_dir: str | None = None
) -> (pd.Series, float, dict)
```

Parameters

Parameter	Type	Default	Description
df	DataFrame	-	Standardized numeric dataframe from data preparation step
target_variable	str or None	None	Variable to exclude from KMO calculation
output_dir	str or None	None	Directory to save KMO results as JSON

Returns

1. kmo_per_variable (pd.Series)
- KMO value for each variable

◦ Index: variable names

◦ Values: KMO scores (0-1)
2. kmo_model (float)
- Overall KMO for the entire dataset

◦ Single value (0-1)

◦ **Decision criterion:** If < 0.50, stop analysis
3. metadata (dict)
- Interpretation of KMO values

◦ Threshold definitions

◦ Explanatory notes

Example Output

Per-Variable KMO:

AGE	0.7423	(Good)
TENURE	0.6641	(Moderate)
INCOME	0.7288	(Good)
BALANCE	0.8105	(Very Good)

Overall KMO:


```
kmo_model = 0.7414 (Good)
```

Decision: $KMO > 0.50 \rightarrow$ Proceed with factor analysis

KMO Calculation Steps (Behind the Scenes)

1. Calculate Correlation Matrix
 - Pearson correlation between all variable pairs
2. Calculate Inverse (Precision Matrix)
 - Used to derive partial correlations
3. Calculate Partial Correlations
 - Relationship between variables after removing other effects
4. Square Both Matrices
 - Focus on magnitude, not direction
5. Clear Diagonals
 - Ignore self-correlations (always 1)
6. Compute KMO
 - Per variable: sum by rows
 - Overall: sum all elements

Usage Example

```
from factor_analysis import calculate_kmo_manual

# Calculate KMO (after data preparation)
kmo_per_var, kmo_model, kmo_meta = calculate_kmo_manual(
    df=df_prepared,
    target_variable="TARGET",
    output_dir="outputs/factor_analysis"
)

# Check overall KMO
print(f"Overall KMO: {kmo_model:.4f}")
if kmo_model < 0.50:
    print("Dataset not suitable for factor analysis")
else:
    print("Proceed with factor analysis")

# Review per-variable KMO
print("\nPer-Variable KMO:")
print(kmo_per_var)
```

```
# Identify weak variables
weak_vars = kmo_per_var[kmo_per_var < 0.50]
if len(weak_vars) > 0:
    print(f"\nVariables with KMO < 0.50: {weak_vars.index.tolist()}")
```

Quality Gate Logic

The KMO test acts as a **quality gate** in the pipeline:



Key Insights

- 1. **KMO > 0.70** is generally considered the sweet spot for factor analysis
- 2. **KMO per variable** helps identify problematic variables that could be removed
- 3. **Partial correlations** reveal if relationships are due to common factors (good) or unique interactions (bad)
- 4. KMO assumes **linear relationships** between variables
- 5. KMO is **not** a test of statistical significance, but a measure of data suitability

Common Issues

Issue	Cause	Solution
KMO < 0.50	Variables too independent	Remove unrelated variables, collect more correlated data
Variable KMO << Overall KMO	One variable ruins structure	Consider removing that variable

Issue	Cause	Solution
Many partial correlations high	Unique factor relationships	Variables have different underlying structures

References

- [MetricGate KMO Documentation](#)
- Kaiser, H. F. (1974). "An index of factorial simplicity"

Helper Function: KMO Interpretation (`interpret_kmo`)

The `interpret_kmo` function provides a human-readable interpretation of KMO values:

```
from factor_analysis import interpret_kmo

# Interpret KMO values
interpretation = interpret_kmo(0.75) # Returns: "Good"
interpretation = interpret_kmo(0.45) # Returns: "Not suitable"
```

Interpretation Scale:

KMO Value	Interpretation
< 0.50	Not suitable
0.50 - 0.60	Weak
0.60 - 0.70	Moderate
0.70 - 0.80	Good
0.80 - 0.90	Very good
≥ 0.90	Excellent

Step 3: Bartlett's Test of Sphericity (`calculate_bartlett_manual`)

Purpose

Bartlett's Test of Sphericity determines whether the correlation matrix is significantly different from an **identity matrix**. It answers the question: "Are the variables sufficiently correlated to justify factor analysis?"

This test acts as a second quality gate. Even if KMO is high, Bartlett's test must also pass (p-value < 0.05).

Why Bartlett's Test Matters

An **identity matrix** has all 1s on the diagonal and 0s everywhere else:

Identity Matrix (No Correlations):			
	AGE	TENURE	INCOME
AGE	1	0	0
TENURE	0	1	0
INCOME	0	0	1

This represents a scenario where variables are completely independent. Bartlett's test checks if your data's correlation matrix is significantly different from this independence structure.

If variables are independent, factor analysis is meaningless because there are no common factors to extract.

The Hypothesis Test

Null Hypothesis (H0): The correlation matrix is an identity matrix (variables are independent, no common factors exist).

Alternative Hypothesis (H1): The correlation matrix is NOT an identity matrix (variables are correlated, common factors exist).

Decision Rule:

- If p-value < 0.05: Reject H0 (continue with factor analysis)
- If p-value >= 0.05: Fail to reject H0 (stop, factor analysis not recommended)

Bartlett's Test Formula

The test statistic is:

$$\chi^2 = -(n - 1 - \frac{2p + 5}{6}) \times \ln(\det(R))$$

Where:

- n = number of samples (rows)
- p = number of variables (columns)
- $\det(R)$ = determinant of correlation matrix
- $\ln$ = natural logarithm

Components Explained:

1. **Determinant of Correlation Matrix** ($\det(R)$)
 - Ranges from 0 to 1
 - $\det(R) = 1$: Variables completely independent (identity matrix)
 - $\det(R) \approx 0$: Variables highly correlated/collinear
 - $\ln(\det(R))$ is negative (we add the negative sign to make chi-square positive)
2. **Correction Factor** ($n - 1 - \frac{2p + 5}{6}$)
 - Adjusts for sample size and number of variables
 - Improves accuracy for small samples or many variables

- Higher correction factor = larger chi-square value

3. Chi-Square Value (χ^2)

- Test statistic follows chi-square distribution
- Compared to critical value with DF degrees of freedom
- Converted to p-value using chi-square CDF

4. Degrees of Freedom

$$DF = \frac{p(p-1)}{2}$$

- Example: with 5 variables, $DF = 5 \times 4 / 2 = 10$

Interpretation Table

p-value	Decision	Action
< 0.001	Strongly reject H0	Continue with factor analysis
0.001 - 0.01	Reject H0	Continue with factor analysis
0.01 - 0.05	Reject H0	Continue with factor analysis
0.05	Borderline	Caution advised
> 0.05	Fail to reject H0	Stop, use factor analysis not recommended

Function Signature

```
calculate_bartlett_manual(
    df: pd.DataFrame,
    target_variable: str | None = None,
    output_dir: str | None = None
) -> (float, float, int, dict)
```

Parameters

Parameter	Type	Default	Description
<code>df</code>	DataFrame	-	Standardized numeric dataframe from data preparation
<code>target_variable</code>	str or None	None	Variable to exclude from Bartlett test
<code>output_dir</code>	str or None	None	Directory to save results as JSON

Returns

1. `chi_square_value` (float)

- The test statistic

- Larger values indicate correlation matrix deviates from identity
- Range: [0, infinity)

2. **p_value** (float)

- Probability under null hypothesis
- Range: [0, 1]
- Decision criterion: if < 0.05 , reject independence

3. **degrees_of_freedom** (int)

- $DF = p(p-1)/2$ where p is number of variables

4. **metadata** (dict)

- Test name and definition
- All calculated values
- Interpretation thresholds
- Suggested action

Example Output

Test Results:

Chi-square value: 85.1200

P-value: 0.000001

Degrees of freedom: 15

Decision: Reject H_0

Interpretation: Factor analysis can continue ($p\text{-value} < 0.05$)

Detailed Calculation Example

Given a dataset with:

- $n = 100$ samples
- $p = 5$ variables
- $\det(R) = 0.1234$

Step 1: Calculate Correction Factor

$$CF = 100 - 1 - \frac{2(5) + 5}{6} = 99 - \frac{15}{6} = 99 - 2.5 = 96.5$$

Step 2: Calculate $\ln(\det(R))$

$$\ln(0.1234) = -2.0910$$

Step 3: Calculate Chi-Square

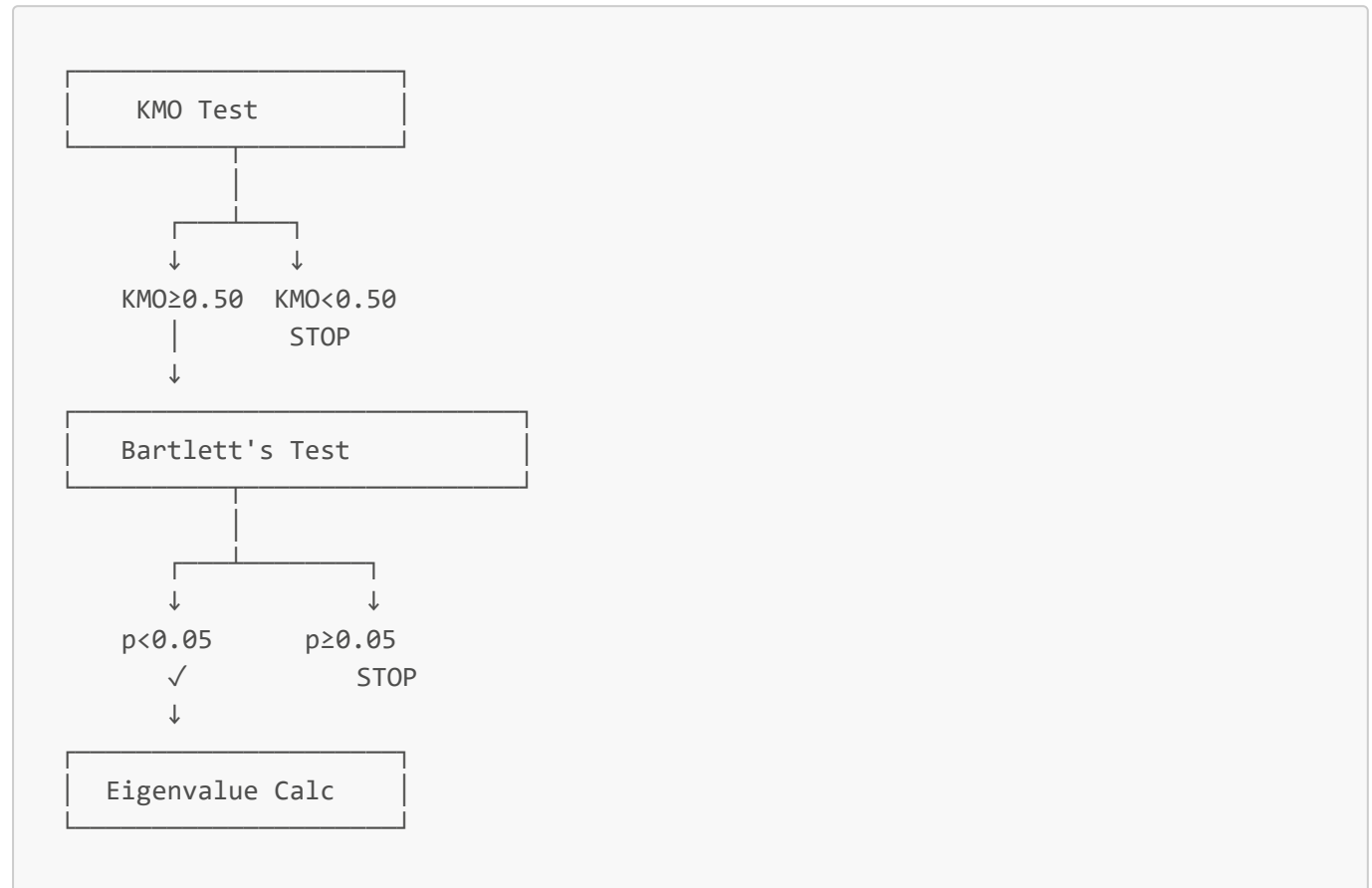
$$\chi^2 = -96.5 \times (-2.0910) = 201.78$$

Step 4: Calculate p-value With $DF = 5(4)/2 = 10$, use chi-square CDF:

$$p\text{-value} = P(\chi_{10}^2 > 201.78) \approx < 0.001$$

Step 5: Decision $p\text{-value} < 0.05$, so reject H_0 . Proceed with factor analysis.

Quality Gate Logic



Both gates must pass for factor analysis to proceed.

Common Scenarios

Scenario	KMO	Bartlett p	Result
Strong structure	0.80	0.00001	Factor analysis recommended
Weak structure	0.45	0.200	Both gates fail, stop
Moderate + marginal	0.65	0.06	KMO passes, Bartlett borderline
Strong but independent	0.15	0.800	KMO fails, variables independent

Why Determinant Matters

The determinant of a correlation matrix indicates multicollinearity:

- **det(R) = 1**: No correlations (identity matrix)
- **det(R) = 0.5**: Moderate correlations (ideal for factor analysis)
- **det(R) = 0.01**: Very high correlations (possible multicollinearity)

- **$\det(R) \approx 0$** : Perfect multicollinearity (variables are linear combinations)

Problems if $\det(R) \leq 0$:

- Cannot take natural logarithm of zero or negative numbers
- Indicates variables are perfectly collinear
- Correlation matrix is singular (non-invertible)
- Solution: Remove redundant variables

Usage Example

```
from factor_analysis import calculate_bartlett_manual

# Run Bartlett's test (after KMO passes)
chi2, p_val, df, bartlett_meta = calculate_bartlett_manual(
    df=df_prepared,
    target_variable="TARGET",
    output_dir="outputs/factor_analysis"
)

# Check results
print(f"Chi-square: {chi2:.4f}")
print(f"P-value: {p_val:.8f}")
print(f"Degrees of freedom: {df}")

if p_val < 0.05:
    print("Decision: Reject H0")
    print("Interpretation: Variables are sufficiently correlated")
    print("Action: Proceed to eigenvalue calculation")
else:
    print("Decision: Fail to reject H0")
    print("Interpretation: Variables are independent")
    print("Action: Factor analysis not recommended")
```

Key Insights

1. **p-value < 0.05 is required** to pass the quality gate
2. **Very small p-values (< 0.001)** indicate strong correlations
3. **Determinant close to 1** suggests weak correlations (high identity matrix similarity)
4. **Determinant close to 0** suggests strong correlations (good for factor analysis)
5. Large sample sizes increase chi-square values (larger datasets detect small correlations)
6. Bartlett's test is sensitive to sample size - with large N, even weak correlations become significant

Common Issues

Issue	Cause	Solution
p-value > 0.05	Variables too independent	Include more correlated variables

Issue	Cause	Solution
det(R) <= 0	Perfect multicollinearity	Remove redundant variables
Very large chi2	Very high correlations	Check for data quality issues
p-value conflicts with KMO	Opposite structures	Investigate variable relationships

References

- [MetricGate Bartlett Test Documentation](#)
- Bartlett, M. S. (1954). "A note on the multiplying factors for various chi square approximations"

Helper Function: Bartlett Interpretation (`interpret_bartlett`)

The `interpret_bartlett` function provides a human-readable interpretation of Bartlett test results:

```
from factor_analysis import interpret_bartlett

# Interpret Bartlett p-values
interpretation, action = interpret_bartlett(0.000001) # Strongly reject H0
interpretation, action = interpret_bartlett(0.10)     # Fail to reject H0

# Returns tuple:
# ("Reject H0 with p-value=0.000001...", "Factor analysis can continue.")
# ("Fail to reject H0 with p-value=0.10...", "Factor analysis is not
# recommended.")
```

Decision Rules:

p-value	Result	Interpretation	Action
< 0.001	Reject H0	Strongly reject independence assumption	Continue analysis
0.001 - 0.05	Reject H0	Reject independence assumption	Continue analysis
>= 0.05	Fail to reject H0	Cannot reject independence	Do NOT continue

Custom Threshold:

```
# Use different p-value threshold if needed (default is 0.05)
interpretation, action = interpret_bartlett(
    p_value=0.03,
    reject_threshold=0.01 # Stricter threshold
)
```

Step 3.5: Quality Gate Validation (`validate_factor_analysis_suitability`)

Purpose

This function acts as an automated quality gate that validates whether the dataset passes both the KMO and Bartlett tests before proceeding with factor extraction. It consolidates the two quality checks and provides clear pass/fail decisions with detailed logging.

Why Quality Gates Matter

The pipeline includes two critical tests that must **both pass** before factor analysis can proceed:

- 1. **KMO Test:** Measures variable intercorrelation (does shared variance exist?)
- 2. **Bartlett Test:** Measures correlation significance (are correlations meaningful?)

Both tests are necessary:

- KMO alone can't determine significance
- Bartlett alone can't assess correlation strength
- Together they validate the data's suitability for factor analysis

Function Signature

```
validate_factor_analysis_suitability(  
    kmo_model: float,  
    bartlett_p_value: float,  
    min_kmo: float = 0.50,  
    max_bartlett_p_value: float = 0.05,  
    output_dir: str | None = None  
) -> dict
```

Parameters

Parameter	Type	Default	Description
kmo_model	float	-	Overall KMO value from KMO test
bartlett_p_value	float	-	P-value from Bartlett's test
min_kmo	float	0.50	Minimum acceptable KMO threshold
max_bartlett_p_value	float	0.05	Maximum acceptable p-value threshold
output_dir	str or None	None	Directory to save validation results as JSON

Returns

metadata_of_validation (dict)

- KMO and Bartlett values
- Threshold settings used
- Pass/fail status for each test

- Overall suitability determination
- Saved to `factor_analysis_suitability_validation.json`

Decision Logic

```
If KMO >= 0.50 AND p-value < 0.05:  
    ✓ PASS - Proceed to eigenvalue calculation  
  
If KMO < 0.50:  
    ✗ FAIL - Dataset has inadequate variable intercorrelation  
  
If p-value >= 0.05:  
    ✗ FAIL - Variables are not sufficiently correlated  
  
If both fail:  
    ✗ FAIL - Dataset not suitable for factor analysis
```

Logging Output

The function provides detailed logging:

```
[VALID] DATASET IS SUITABLE FOR FACTOR ANALYSIS  
KMO Model: 0.741234 (meets minimum requirement of 0.50)  
Bartlett's p-value: 0.00000001 (below maximum threshold of 0.05)  
Factor extraction can proceed with confidence.
```

Or in case of failure:

```
[WARNING] KMO TEST FAILED - INADEQUATE SAMPLING ADEQUACY  
KMO=0.42 is BELOW the minimum threshold of 0.50.  
Factor analysis is NOT RECOMMENDED...
```

Usage Example

```
from factor_analysis import calculate_kmo_manual, calculate_bartlett_manual,  
validate_factor_analysis_suitability  
  
# Run KMO and Bartlett tests  
kmo_per_var, kmo_model, _ = calculate_kmo_manual(df_prepared)  
chi2, p_val, _, _ = calculate_bartlett_manual(df_prepared)  
  
# Validate suitability  
validation_result = validate_factor_analysis_suitability(  
    kmo_model=kmo_model,  
    bartlett_p_value=p_val,
```

```
        min_kmo=0.50,
        max_bartlett_p_value=0.05,
        output_dir="outputs/factor_analysis"
    )

    # Check results
    if validation_result['isSuitableForFactorAnalysis'] == 'pass':
        print("Factor analysis can proceed")
        proceed_to_eigenvalues()
    else:
        print("Dataset not suitable - alternative methods recommended")
```

Threshold Customization

You can adjust thresholds based on your use case:

More Strict Analysis (higher standards):

```
validate_factor_analysis_suitability(
    kmo_model=0.70, # Require "Good" KMO, not just "Weak"
    bartlett_p_value=0.001, # p-value must be highly significant
    min_kmo=0.70,
    max_bartlett_p_value=0.001
)
```

More Permissive Analysis (relaxed criteria):

```
validate_factor_analysis_suitability(
    kmo_model=0.50, # Accept "Weak" KMO
    bartlett_p_value=0.10, # Accept p-value up to 0.10
    min_kmo=0.50,
    max_bartlett_p_value=0.10
)
```

Common Scenarios

KMO	p-value	Result	Action
0.75	0.000001	PASS	Continue to eigenvalues
0.40	0.000001	FAIL	Remove unrelated variables
0.75	0.08	FAIL	Correlations weak, try different variables
0.45	0.08	FAIL	Both gates fail, use alternative methods

Step 4: Eigenvalue Calculation (`calculate_eigenvalues`)

Purpose

This function calculates eigenvalues and eigenvectors from the correlation matrix. These values determine how many factors to retain and how much variance each factor explains.

Eigenvalues answer the question: "How much of the data's total variation is captured by each factor?"

Why Eigenvalues Matter

Factor analysis reduces hundreds of variables into a smaller number of underlying factors. But how many factors are needed?

Eigenvalues tell us:

- How much variance each factor explains
- Whether factors are worth keeping
- When we have captured enough information

Without eigenvalues, we wouldn't know how many factors to extract or whether we're losing important information.

Eigenvalues and Eigenvectors Explained

Eigenvalue (λ):

- A scalar number representing variance explained by one factor
- Ranges from 0 to p (number of variables)
- For correlation matrix: sum of all eigenvalues equals p
- Larger eigenvalues indicate more important factors

Eigenvector (v):

- A direction vector showing how variables contribute to a factor
- Has length equal to number of variables
- Components show which original variables influence the factor
- Later used to calculate factor loadings

Relationship:

$$Rv = \lambda v$$

Where R is the correlation matrix, v is eigenvector, and lambda is eigenvalue.

Mathematical Foundation

For a correlation matrix R, we solve the characteristic equation:

$$\det(R - \lambda I) = 0$$

This yields:

- p eigenvalues (one for each variable)

- p eigenvectors (each corresponding to an eigenvalue)

Key Properties:

1. **Eigenvalues are real** (correlation matrix is symmetric)
2. **Eigenvalues are non-negative** (correlation matrix is positive semi-definite)
3. **Sum of eigenvalues = p** (number of variables)
4. **Eigenvectors are orthogonal** (perpendicular to each other)

Explained Variance

Variance Explained Ratio:

$$\text{Variance}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$$

Cumulative Variance:

$$\text{Cumulative}_k = \sum_{i=1}^k \text{Variance}_i$$

Example with 5 variables:

Factor	Eigenvalue	Variance %	Cumulative %
1	3.42	68.4	68.4
2	1.08	21.6	90.0
3	0.28	5.6	95.6
4	0.18	3.6	99.2
5	0.04	0.8	100.0

Retain Factors 1 and 2 (eigenvalue > 1), which capture 90% of variance.

Kaiser Criterion (Default Rule)

Rule: Retain factors with eigenvalue > 1

Rationale:

- Eigenvalue = 1 means factor explains as much variance as one original variable
- Eigenvalue < 1 means factor explains less than one variable
- For standardized variables, this is a reasonable threshold

Example:

```
Eigenvalue 3.42 > 1: Retain  
Eigenvalue 1.08 > 1: Retain  
Eigenvalue 0.28 < 1: Drop
```

Result: Keep 2 factors instead of 5.

Alternative Selection Rules

1. Cumulative Variance Threshold

- Retain factors until cumulative variance reaches target (usually 80-95%)
- More data-driven than Kaiser criterion

2. Scree Plot Method

- Plot eigenvalues in descending order
- Look for "elbow" where slope changes
- Retain factors before the elbow

3. Eigenvalue Threshold

- Set custom threshold (e.g., 1.2, 1.5)
- More conservative than Kaiser criterion

Function Signature

```
calculate_eigenvalues(  
    df: pd.DataFrame,  
    target_variable: str | None = None,  
    output_dir: str | None = None  
) -> (pd.DataFrame, np.ndarray, np.ndarray)
```

Parameters

Parameter	Type	Default	Description
<code>df</code>	DataFrame	-	Standardized numeric dataframe from KMO and Bartlett tests
<code>target_variable</code>	str or None	None	Variable to exclude from eigenvalue calculation
<code>output_dir</code>	str or None	None	Directory to save eigenvalues table as JSON

Returns

1. `eigen_table` (DataFrame)

- Columns: factor, eigenvalue, explainedVarianceRatio, cumulativeVarianceRatio
- Shows variance explained by each factor

2. **eigenvalues** (np.ndarray)

- Array of eigenvalues in descending order
- Used to determine number of factors

3. **eigenvectors** (np.ndarray)

- Matrix of eigenvectors (one column per factor)
- Used to calculate factor loadings

Example Output

Eigenvalue Table:

	factor	eigenvalue	explainedVarianceRatio	cumulativeVarianceRatio
0	Factor_1	3.42	0.6840	0.6840
1	Factor_2	1.08	0.2160	0.9000
2	Factor_3	0.28	0.0560	0.9560
3	Factor_4	0.18	0.0360	0.9920
4	Factor_5	0.04	0.0080	1.0000

Interpretation:

- Factors 1-2 have eigenvalue > 1, retain them
- They explain 90% of total variance
- 3 variables worth of information compressed into 2 factors
- Dimensionality reduction: 5 → 2 variables

Calculation Steps

1. Calculate Correlation Matrix
 - Pearson correlation between all variable pairs
2. Solve Characteristic Equation
 - $\det(R - \lambda I) = 0$
 - Find eigenvalues and eigenvectors
3. Take Real Parts
 - Numerical computation may have tiny imaginary components
 - Extract real values
4. Sort in Descending Order
 - Arrange by eigenvalue magnitude (largest first)
 - Reorder eigenvectors to match
5. Calculate Variance Explained

- Eigenvalue / Sum of all eigenvalues
- Cumulative sum for total variance

6. Apply Retention Rules

- Kaiser: eigenvalue > 1
- Or other thresholds

Detailed Calculation Example

Given correlation matrix R (3 variables):

$$R = \begin{bmatrix} 1.00 & 0.80 & 0.60 \\ 0.80 & 1.00 & 0.70 \\ 0.60 & 0.70 & 1.00 \end{bmatrix}$$

Step 1: Solve $\det(R - \lambda I) = 0$ This yields eigenvalues (approximated):

- $\lambda_1 = 2.40$
- $\lambda_2 = 0.42$
- $\lambda_3 = 0.18$

Step 2: Sort descending Already in order: 2.40, 0.42, 0.18

Step 3: Calculate explained variance

- Total variance = $2.40 + 0.42 + 0.18 = 3.00$
- Variance $\lambda_1 = 2.40 / 3.00 = 0.800$ (80%)
- Variance $\lambda_2 = 0.42 / 3.00 = 0.140$ (14%)
- Variance $\lambda_3 = 0.18 / 3.00 = 0.060$ (6%)

Step 4: Cumulative variance

- Factor 1: 80%
- Factors 1-2: 94%
- Factors 1-3: 100%

Step 5: Apply Kaiser criterion

- $\lambda_1 = 2.40 > 1$: Retain
- $\lambda_2 = 0.42 < 1$: Drop
- $\lambda_3 = 0.18 < 1$: Drop

Result: Retain 1 factor, explaining 80% of variance.

Usage Example

```
from factor_analysis import calculate_eigenvalues
```

```

# Calculate eigenvalues and eigenvectors
eigen_table, eigenvalues, eigenvectors = calculate_eigenvalues(
    df=df_prepared,
    target_variable="TARGET",
    output_dir="outputs/factor_analysis"
)

# Review eigenvalue table
print("Eigenvalue Analysis:")
print(eigen_table)

# Count factors by Kaiser criterion
n_factors_kaiser = (eigenvalues > 1).sum()
print(f"\nFactors with eigenvalue > 1: {n_factors_kaiser}")

# Check cumulative variance
cumulative_var = eigen_table['cumulativeVarianceRatio'].values
print(f"Variance explained by Kaiser factors: {cumulative_var[n_factors_kaiser - 1]:.1%}")

# Alternative: 80% variance rule
var_80_idx = (cumulative_var >= 0.80).argmax()
print(f"Factors needed for 80% variance: {var_80_idx + 1}")

# Alternative: 95% variance rule
var_95_idx = (cumulative_var >= 0.95).argmax()
print(f"Factors needed for 95% variance: {var_95_idx + 1}")

```

Kaiser Criterion Detailed

The Kaiser criterion (eigenvalue > 1) is the default for good reasons:

1. Intuitive interpretation

- 1 eigenvalue = 1 original variable's worth of variance
- Keep only factors at least as important as one variable

2. Statistical basis

- Average eigenvalue for p variables = 1
- Eigenvalue > 1 means above average importance

3. Practical balance

- Not too conservative (keeps useful factors)
- Not too aggressive (avoids noise factors)

4. When to override:

- Cumulative variance < 70% with Kaiser rule → lower threshold
- Too many factors extracted → raise threshold to 1.2 or 1.5
- Domain knowledge suggests different number

Variance Thresholds

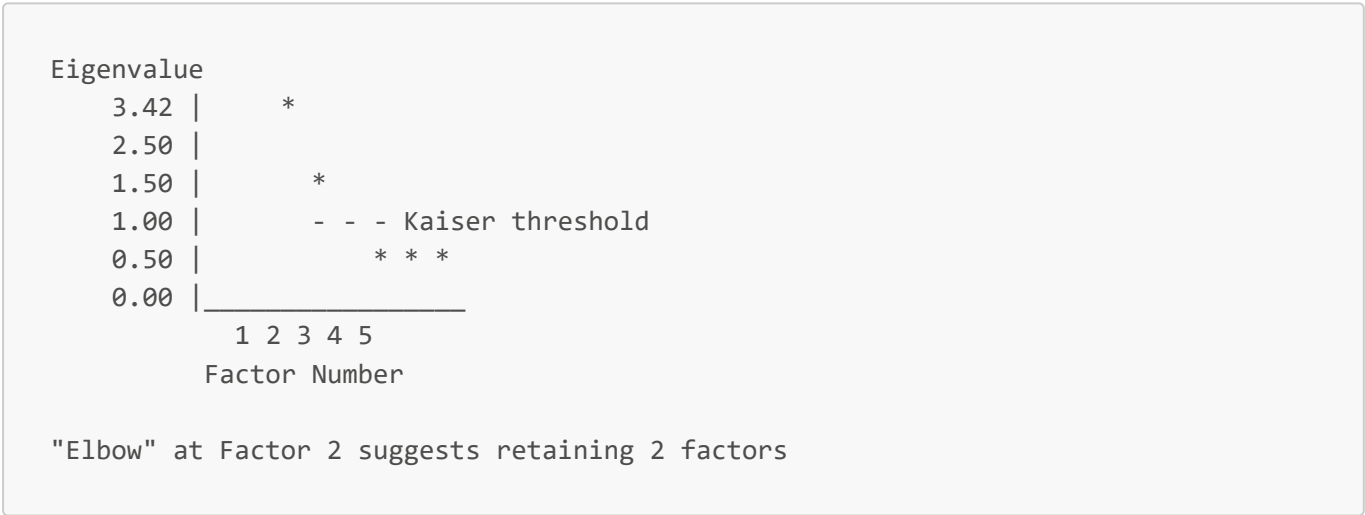
Common benchmarks:

Threshold	Interpretation	Use Case
50%	Minimum viable	Exploratory analysis
70%	Standard practice	Most factor analyses
80%	High information retention	Critical applications
90%	Very comprehensive	When information loss is costly
95%	Near-complete retention	Quality-focused analysis

Kaiser criterion typically results in 70-80% variance retained.

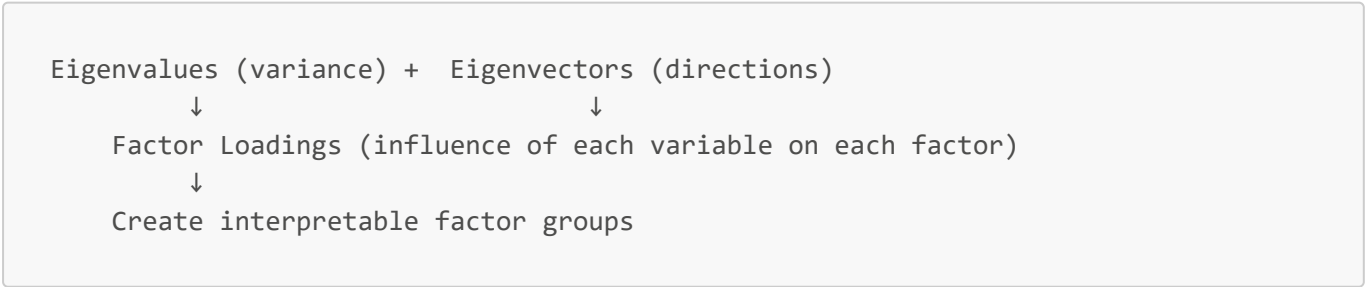
Scree Plot Interpretation

While this function doesn't create plots, the eigenvalue data enables one:



Relationship to Next Steps

Eigenvalues → Eigenvectors → Factor Loadings:



Example:

- Eigenvalue 3.42 tells us Factor 1 is important
- Eigenvector [0.6, 0.7, 0.5] tells us which variables drive Factor 1
- Together: Factor 1 = 0.6VAR1 + 0.7VAR2 + 0.5*VAR3

Common Issues and Solutions

Issue	Cause	Solution
All eigenvalues < 1	Variables independent	Use KMO/Bartlett results; check data quality
Eigenvalue = 0	Perfect multicollinearity	Remove redundant variables
One large eigenvalue	Variables form single cluster	May need only 1 factor
Many eigenvalues near 1	Multiple equally important factors	Use variance threshold instead

Key Insights

1. **Dimensionality reduction** is the main benefit - compress 100 variables to 5-10 factors
2. **Kaiser criterion** is standard but not always optimal - consider variance thresholds
3. **Eigenvalues sum to p** - this is a useful sanity check
4. **Retention decisions matter** - affect downstream factor loadings and interpretation
5. **Eigenvectors are orthogonal** - factors are uncorrelated (important for interpretation)
6. **Larger datasets** don't change eigenvalue structure, just statistical significance

References

- [MetricGate Eigenvalue Documentation](#)
- Kaiser, H. F. (1960). "The application of electronic computers to factor analysis"

Step 5: Factor Loadings (`calculate_factor_loadings`)

Purpose

Factor loadings show how strongly each original variable is associated with each extracted factor. They bridge the gap between abstract factors and interpretable variables.

A factor loading is essentially a correlation between a variable and a factor, indicating how much that variable contributes to defining the factor.

Why Factor Loadings Matter

After extracting factors (via eigenvalues), we need to understand what they represent. Factor loadings answer:

- Which variables define Factor 1?
- How important is variable X to Factor 2?
- Can we interpret this factor based on its variables?

Without loadings, factors are just numbers. With loadings, they become interpretable concepts.

Factor Loadings Formula

The core calculation is:

$$\text{Loading}_{ij} = \text{eigenvector}_{ij} \times \sqrt{\lambda_j}$$

Where:

- eigenvector_{ij} = direction of factor j in variable i space
- λ_j = eigenvalue of factor j
- $\sqrt{\lambda_j}$ = strength of factor j

Why Multiply by Square Root of Eigenvalue?

1. **Eigenvector** = unit vector (length 1), pure direction
2. **Eigenvalue** = variance magnitude (unscaled)
3. **sqrt(eigenvalue)** = converts between eigenvalue scale and correlation scale
4. **Product** = combines direction and strength into meaningful loading

Result: Loading is similar to a correlation coefficient, ranging from approximately -1 to +1.

Interpretation of Loadings

General Rules:

Loading Range	Interpretation
0.90 to 1.00	Very strong positive association
0.70 to 0.89	Strong positive association
0.50 to 0.69	Moderate positive association
0.30 to 0.49	Weak positive association
-0.30 to 0.30	Negligible association
-0.49 to -0.30	Weak negative association
-0.69 to -0.50	Moderate negative association
-0.89 to -0.70	Strong negative association
-1.00 to -0.90	Very strong negative association

Example Interpretation:

	Factor_1	Factor_2	Factor_3
AGE	0.92	-0.08	0.15
INCOME	0.88	0.12	0.20
TENURE	0.85	0.18	-0.10
SPENDING	-0.05	0.91	-0.08
TRANSACTIONS	-0.12	0.87	0.14
INQUIRIES	-0.08	0.10	0.95
CREDIT_UTIL	0.15	0.22	0.88

Interpretation:

- Factor 1: Dominated by AGE, INCOME, TENURE (demographic/stability factor)
- Factor 2: Dominated by SPENDING, TRANSACTIONS (activity factor)
- Factor 3: Dominated by INQUIRIES, CREDIT_UTIL (credit behavior factor)

Unrotated vs. Rotated Loadings

Unrotated Loadings (Principal Component Loadings):

- Direct calculation from eigenvectors and eigenvalues
- First factor captures maximum variance
- Later factors capture remaining variance
- Generally less interpretable (variables load on multiple factors)

Rotated Loadings (after Varimax):

- Loadings are transformed to improve interpretability
- Each variable tends to load high on fewer factors
- Easier to assign variables to factor groups
- Total variance explained stays the same, only structure changes

Varimax Rotation

Purpose: Simplify factor loadings by rotating the factor axes to maximize variance of loadings within each factor.

What It Does:

- Pushes loadings toward either high or low values
- Makes clear which variables belong to which factor
- Improves factor interpretation

Example Before Rotation:

	Factor_1	Factor_2
Variable_A	0.62	0.48
Variable_B	0.60	0.51
Variable_C	0.55	0.58
Variable_D	0.49	0.61

All variables load moderately on both factors (ambiguous).

Example After Varimax Rotation:

	Factor_1	Factor_2
Variable_A	0.90	0.10
Variable_B	0.85	0.15

Variable_C	0.10	0.88
Variable_D	0.15	0.84

Clear separation: A,B belong to Factor 1; C,D belong to Factor 2.

Why Varimax?

Varimax maximizes:

$$V = \frac{1}{p} \sum_{j=1}^m \sum_{i=1}^p a_{ij}^4 - \left(\frac{1}{p} \sum_{j=1}^m \sum_{i=1}^p a_{ij}^2 \right)^2$$

This objective function favors loadings near 0 or near +/-1, creating simpler structure.

Key Properties:

- Rotation preserves total variance explained
- Only changes loadings structure, not factors themselves
- Improves interpretability without information loss
- Orthogonal rotation keeps factors uncorrelated

Function Signature

```
calculate_factor_loadings(  
    df: pd.DataFrame,  
    eigenvalues: np.ndarray,  
    eigenvectors: np.ndarray,  
    rotation: str = "varimax",  
    eigenvalue_selection_method: str | None = None,  
    eigenvalue_threshold: float = 1.0,  
    target_variable: str | None = None,  
    output_dir: str | None = None  
) -> (pd.DataFrame, int)
```

Parameters

Parameter	Type	Default	Description
df	DataFrame	-	Standardized numeric dataframe
eigenvalues	ndarray	-	Eigenvalues in descending order from Step 4
eigenvectors	ndarray	-	Eigenvectors in descending order from Step 4
rotation	str	"varimax"	Rotation method: "varimax" or None

Parameter	Type	Default	Description
<code>eigenvalue_selection_method</code>	str or None	None	Method for selecting factors: "kaiser", "number", "variance", or None (all factors)
<code>eigenvalue_threshold</code>	float	1.0	Threshold for factor retention (interpretation depends on selection method)
<code>target_variable</code>	str or None	None	Variable to exclude from calculation
<code>output_dir</code>	str or None	None	Directory to save loadings as JSON

Returns

1. `loadings_df` (DataFrame)

- Rows: variable names
- Columns: Factor_1, Factor_2, etc.
- Values: factor loadings

2. `n_factors` (int)

- Number of factors retained
- Based on eigenvalue threshold

Example Output

Before Rotation:

	Factor_1	Factor_2
AGE	0.6234	0.5891
INCOME	0.6012	0.6145
TENURE	0.5847	0.5123
SPENDING	-0.4562	0.7234
TRANSACTIONS	-0.4189	0.7156

After Varimax Rotation:

	Factor_1	Factor_2
AGE	0.8934	0.1289
INCOME	0.8745	0.2013
TENURE	0.8612	0.1567
SPENDING	-0.0923	0.9145
TRANSACTIONS	-0.1245	0.8867

Factor Selection Methods

`eigenvalue_selection_method` Options:

Method	Description	<code>eigenvalue_threshold</code> Meaning
None	Retain all factors	Not used (ignored)
"kaiser"	Retain factors with eigenvalue > threshold	Minimum eigenvalue (default: 1.0)
"number"	Retain specific number of factors	Number of factors to retain
"variance"	Retain factors explaining cumulative variance	Cumulative variance threshold (e.g., 0.80 for 80%)

Examples:

- `eigenvalue_selection_method="kaiser", eigenvalue_threshold=1.0` → Retain factors with eigenvalue > 1 (default)
- `eigenvalue_selection_method="number", eigenvalue_threshold=5` → Retain exactly 5 factors
- `eigenvalue_selection_method="variance", eigenvalue_threshold=0.80` → Retain factors explaining ≥80% variance
- `eigenvalue_selection_method=None` → Retain all factors

Calculation Steps

Step 1: Select Retained Factors

- Use `eigenvalue_selection_method` to determine which factors to keep
- Apply `eigenvalue_threshold` according to the selected method
- Get corresponding eigenvalues and eigenvectors

Step 2: Calculate Unrotated Loadings

- $\text{loading} = \text{eigenvector} \times \sqrt{\text{eigenvalue}}$
- Creates loading matrix (variables × factors)

Step 3: Apply Rotation (Optional)

- Varimax: maximize variance of squared loadings
- None: use unrotated loadings

Step 4: Create DataFrame

- Rows: variable names
- Columns: Factor_1, Factor_2, etc.
- Values: loadings (correlation-like values)

Detailed Calculation Example

Given from Step 4:

- Eigenvalues: [3.42, 1.08, 0.28, ...]

- Eigenvectors: 3x3 matrix (3 variables, showing first 2 factors)

Step 1: Select Factors with eigenvalue > 1

- Factor 1: eigenvalue = 3.42
- Factor 2: eigenvalue = 1.08
- Drop factors 3+

Step 2: Extract eigenvectors for selected factors

```
Eigenvectors (full):      Selected (first 2):
[0.577  0.707  ...]      [0.577  0.707]
[0.577 -0.707  ...]      [0.577 -0.707]
[0.577  0.000  ...]      [0.577  0.000]
```

Step 3: Calculate loadings

```
sqrt(3.42) = 1.849
sqrt(1.08) = 1.039

Loading_1,1 = 0.577 × 1.849 = 1.067
Loading_1,2 = 0.707 × 1.039 = 0.735
Loading_2,1 = 0.577 × 1.849 = 1.067
Loading_2,2 = -0.707 × 1.039 = -0.735
Loading_3,1 = 0.577 × 1.849 = 1.067
Loading_3,2 = 0.000 × 1.039 = 0.000
```

```
Loadings Matrix:
          Factor_1  Factor_2
VAR_1      1.067    0.735
VAR_2      1.067   -0.735
VAR_3      1.067    0.000
```

Step 4: Apply Varimax Rotation

- Iteratively rotates factor axes
- Converges when improvement < tolerance
- Result: simpler, more interpretable structure

Sum of Squared Loadings

An important property: Sum of squared loadings per factor equals the eigenvalue.

$$\sum_{i=1}^p \text{loading}_{ij}^2 = \lambda_j$$

This is a useful sanity check:

Loadings for Factor_1: [0.92, 0.88, 0.85, -0.05, -0.12, -0.08]
 Sum of squares: $0.92^2 + 0.88^2 + 0.85^2 + 0.05^2 + 0.12^2 + 0.08^2$
 $= 0.846 + 0.774 + 0.722 + 0.003 + 0.014 + 0.006$
 $= 2.365$

Should match: Eigenvalue of Factor_1 = 2.365

If this doesn't match, check calculations.

Usage Example

```
from factor_analysis import calculate_factor_loadings

# Calculate factor loadings (after eigenvalue calculation)
loadings_df, n_factors = calculate_factor_loadings(
    df=df_prepared,
    eigenvalues=eigenvalues,
    eigenvectors=eigenvectors,
    rotation="varimax",
    eigenvalue_threshold=1.0,
    target_variable="TARGET",
    output_dir="outputs/factor_analysis"
)

# Display loadings
print("Factor Loadings (Varimax Rotated):")
print(loadings_df)

# Identify strong loadings (>= 0.70 in absolute value)
strong_loadings = loadings_df.abs() >= 0.70
for factor in loadings_df.columns:
    strong_vars = strong_loadings[factor][strong_loadings[factor]].index.tolist()
    print(f"\n{factor} Strong Variables: {strong_vars}")

# Find primary factor for each variable
primary_factors = loadings_df.abs().idxmax(axis=1)
print("\nPrimary Factor Assignment:")
print(primary_factors)
```

Interpreting Complex Loadings

Case 1: Variable Loads High on Multiple Factors

	Factor_1	Factor_2
VARIABLE_X	0.75	0.68

- Variable is complex, related to multiple concepts
- May represent a cross-cutting theme
- Consider domain knowledge for interpretation

Case 2: Variable Loads Poorly on All Factors

	Factor_1	Factor_2	Factor_3
VARIABLE_Y	0.15	0.18	0.22

- Variable doesn't fit the factor structure
- May be:
 - Measurement error
 - Independent concept
 - Should be excluded

Case 3: Negative Loading

	Factor_1	Factor_2
VARIABLE_Z	-0.92	0.08

- Variable is inversely related to factor
- If Factor_1 increases, VARIABLE_Z decreases
- Important for interpretation (e.g., "risk" vs "safety")

Quality Checks for Loadings

1. Check sum of squared loadings

- Should equal eigenvalue for each factor

2. Check variable coverage

- Most variables should have one strong loading
- Few variables should load equally on multiple factors

3. Check interpretation

- Do loadings tell a coherent story?
- Do high-loading variables make sense together?

4. Check rotation quality

- After Varimax, most variables should have clear primary factor
- Before rotation may show ambiguity

No Rotation vs. Varimax

When to Use No Rotation:

- Analysis of total variance explained (first factor captures maximum)
- Theoretical interest in hierarchical factor importance
- Comparing with PCA results

When to Use Varimax (Recommended):

- Factor interpretation and naming
- Creating factor groups
- Business-focused analysis
- Publication (more interpretable)

Common Loadings Patterns

Pattern	Interpretation	Action
Clear high/low	Distinct factors	Proceed to grouping
Many moderate loadings	Overlapping factors	Review factor count; consider lowering threshold
One dominant loading	Single factor dominates	May not need multiple factors
No strong loadings	Weak structure	Review data quality and KMO/Bartlett results

Loading Thresholds for Grouping

Loading size determines variable importance for next step (grouping):

- **≥ 0.70** : Very strong (definitely belongs to factor)
- **0.50-0.69**: Strong (likely belongs to factor)
- **0.30-0.49**: Moderate (consider contextually)
- **< 0.30** : Weak (usually not assigned)

Default grouping uses ≥ 0.50 .

Relationship to Factor Scores

Note: This function calculates loadings, not factor scores.

- **Loadings** (this step): How original variables define factors
- **Factor Scores** (not in current pipeline): How observations (rows) score on factors

Example:

- Loading tells you "Age strongly defines Factor 1"
- Factor score tells you "Person X scores 1.5 on Factor 1"

Key Insights

1. **Loadings are standardized** (based on standardized data)
2. **Rotation improves interpretability** without losing information
3. **Negative loadings matter** - indicate inverse relationships
4. **Sum of squared loadings = eigenvalue** - always check this

- 5. **Not all variables need to load high** - some may be independent
- 6. **Varimax is standard** for applied factor analysis

References

- [Factor Loadings Technical Guide](#)
- Kaiser, H. F. (1958). "The varimax criterion for analytic rotation in factor analysis"
- Harman, H. H. (1976). "Modern Factor Analysis"

Step 6: Factor Grouping (`create_factor_groups`)

Purpose

The final step converts factor loadings into actionable variable groups. Each variable is assigned to the factor where it loads most strongly, creating interpretable clusters of related variables.

Factor grouping answers: "Which variables define each factor, and should they be treated as a group?"

Why Factor Grouping Matters

After factor loadings, we have numbers but not actionable insights. Grouping transforms loadings into:

- Interpretable factor definitions
- Variable clusters for analysis/modeling
- Decision frameworks for feature selection
- Basis for dimensionality reduction

Without grouping, factor analysis results are incomplete.

Core Concept: Assign Variables to Factors

Each variable is assigned to exactly one factor: the one with its highest absolute loading.

Example:

	Factor_1	Factor_2	Factor_3
AGE	0.92	-0.08	0.15
INCOME	0.88	0.12	0.20
SPENDING	-0.05	0.91	0.14

AGE assigned to Factor_1 ($|0.92|$ is highest)

INCOME assigned to Factor_1 ($|0.88|$ is highest)

SPENDING assigned to Factor_2 ($|0.91|$ is highest)

Variable Assignment

Every variable is assigned to exactly one factor based on maximum absolute loading.

Example:

Variable	Max Loading	Primary Factor	Group Status
AGE	0.92	Factor_1	STRONG_LOADING
INCOME	0.88	Factor_1	STRONG_LOADING
MISC_VAR	0.35	Factor_3	WEAK_LOADING

All variables (both STRONG_LOADING and WEAK_LOADING) are included in the final factor groups.

Loading Threshold Explained

The **loading threshold** classifies variables based on strength of their association with the assigned factor:

- **STRONG_LOADING:** max absolute loading >= threshold (default 0.50)
- **WEAK_LOADING:** max absolute loading < threshold

Common Thresholds:

Threshold	Interpretation	Use Case
0.30	Weak threshold, inclusive	Exploratory analysis
0.40	Moderate threshold	Balanced approach
0.50	Standard threshold	Recommended default
0.60	Strong threshold	Conservative analysis
0.70	Very strong threshold	Strict grouping

Default (0.50) provides balanced classification. Note: All variables are included in groups regardless of threshold.

Function Signature

```
create_factor_groups(  
    loadings_df: pd.DataFrame,  
    loading_threshold: float = 0.50,  
    output_dir: str | None = None  
) -> (pd.DataFrame, pd.DataFrame, dict)
```

Parameters

Parameter	Type	Default	Description
loadings_df	DataFrame	-	Factor loadings from Step 5
loading_threshold	float	0.50	Threshold for classifying variables as STRONG_LOADING (>= threshold) or WEAK_LOADING (< threshold)

Parameter	Type	Default	Description
output_dir	str or None	None	Directory to save grouping results as CSV/JSON

Returns

1. **grouping_table** (DataFrame)
- Rows: one per variable
 - Columns: variable, assignedFactor, maxAbsLoading, loadingValue, groupStatus
 - Detailed variable-level assignments
2. **grouped_summary** (DataFrame)
- Rows: one per factor
 - Columns: assignedFactor, variablesInGroup
 - Lists all variables assigned to each factor (both STRONG_LOADING and WEAK_LOADING)
3. **metadata** (dict)
- Grouping logic explanation
 - Threshold definition
 - Sample data

Example Output

Grouping Table (Detailed):

	variable	assignedFactor	maxAbsLoading	loadingValue	groupStatus
0	AGE	Factor_1	0.92	0.92	STRONG_LOADING
1	INCOME	Factor_1	0.88	0.88	STRONG_LOADING
2	TENURE	Factor_1	0.85	0.85	STRONG_LOADING
3	SPENDING	Factor_2	0.91	0.91	STRONG_LOADING
4	TRANSACTIONS	Factor_2	0.87	0.87	STRONG_LOADING
5	INQUIRIES	Factor_3	0.95	0.95	STRONG_LOADING
6	CREDIT_UTIL	Factor_3	0.88	0.88	STRONG_LOADING
7	MISC_VAR	Factor_1	0.38	0.38	WEAK_LOADING

Grouped Summary (Aggregated - All Variables):

	assignedFactor	variablesInGroup
0	Factor_1	[AGE, INCOME, TENURE, MISC_VAR]
1	Factor_2	[SPENDING, TRANSACTIONS]
2	Factor_3	[INQUIRIES, CREDIT_UTIL]

Note: MISC_VAR is included even though it has WEAK_LOADING status.

Detailed Process

Step 1: Calculate Absolute Loadings

- Take absolute value of all loadings
- Ignore positive/negative direction temporarily

Step 2: Find Maximum per Variable

- For each variable, identify highest absolute loading
- That factor becomes variable's primary assignment

Step 3: Retrieve Signed Loadings

- Get actual loading value (with sign) for primary factor
- Preserves direction information

Step 4: Classify Group Status

- Compare max absolute loading to threshold
- `STRONG_LOADING` if $\geq$ threshold
- `WEAK_LOADING` if $<$ threshold

Step 5: Sort for Clarity

- Sort by: (1) assigned factor, (2) max loading descending
- Groups naturally cluster together

Step 6: Create Summary

- Group all variables by assigned factor
- List all group members for each factor (both `STRONG_LOADING` and `WEAK_LOADING`)

Grouping Table Columns Explained

variable

- Original variable name from input data

assignedFactor

- Factor with highest absolute loading
- Each variable assigned to exactly one factor
- Based on $\max(|\text{loadings}|)$ for that variable

maxAbsLoading

- Absolute value of highest loading
- Ranges: 0 to 1 (approximately)
- Higher = stronger assignment

loadingValue

- Actual loading (with sign) on assigned factor
- Can be positive or negative
- Shows direction of relationship

groupStatus

- Classification: STRONG_LOADING or WEAK_LOADING
- Based on: $\text{maxAbsLoading} \geq \text{loading_threshold}$
- STRONG_LOADING: Variable strongly associated with assigned factor
- WEAK_LOADING: Variable weakly associated with assigned factor
- Note: Both statuses included in final grouped_summary

Interpretation Guide

Variables in Same Group = Similar Meaning

Factor_1 Group: [AGE, INCOME, EMPLOYMENT_YEARS]
Interpretation: Demographics/Experience factor
Concept: Represents customer maturity/stability

Factor_2 Group: [MONTHLY_SPENDING, TRANSACTIONS, ACTIVE_ACCOUNTS]
Interpretation: Activity/Engagement factor
Concept: Represents customer activity level

Factor_3 Group: [CREDIT_INQUIRIES, NEW_ACCOUNTS, RECENT_DELINQUENCY]
Interpretation: Credit Risk/Credit Seeking factor
Concept: Represents recent credit behavior

Understanding WEAK_LOADING Variables

Variables with WEAK_LOADING status (below threshold) are included in groups but indicate weak association:

Possible Meanings:

1. **Independent concept** - Variable doesn't strongly align with any factor
2. **Measurement noise** - Variable may contain random variation
3. **Multi-factor relationship** - Variable relates to multiple factors weakly
4. **Data quality issue** - May warrant investigation

Actions:

- Review in context with factor interpretation
- Investigate data quality if unexpected
- Consider for exclusion in downstream modeling if needed

When WEAK_LOADING Variables Appear:

- Variable doesn't fit the factor structure
- May represent independent concept
- Possible data quality issue
- Could be measurement noise

Negative Loadings in Groups

Variables with negative loadings can belong to groups.

Example:

Factor_1 Group:

- AGE (loading: +0.92)
- RISK_SCORE (loading: -0.88)

Interpretation: Both define Factor_1, but inversely

- Higher AGE → Higher Factor_1
 - Higher RISK_SCORE → Lower Factor_1
- (Older customers have lower risk)

Important: Sign matters for interpretation but not for grouping.

Usage Example

```
from factor_analysis import create_factor_groups

# Create factor groups (after factor loadings)
grouping_table, grouped_summary, grouping_meta = create_factor_groups(
    loadings_df=loadings_df,
    loading_threshold=0.50,
    output_dir="outputs/factor_analysis"
)

# Display detailed assignments
print("Detailed Variable Assignments:")
print(grouping_table)

# Display factor groups
print("\nFactor Groups (Summary):")
print(grouped_summary)

# Analyze group structure
print("\nGroup Statistics:")
for _, row in grouped_summary.iterrows():
    factor = row['assignedFactor']
    variables = row['variablesInGroup']
    print(f"{factor}: {len(variables)} variables")

# Identify weak variables
weak_vars = grouping_table[grouping_table['groupStatus'] == 'WEAK_LOADING']
if len(weak_vars) > 0:
    print(f"\nWeak Variables (Low Association): {weak_vars['variable'].tolist()}")

# Create factor labels based on groups
factor_labels = {}
for _, row in grouped_summary.iterrows():
    factor = row['assignedFactor']
```

```

variables = row['variablesInGroup']
# Manually create descriptive label
if factor == 'Factor_1':
    factor_labels[factor] = 'Stability/Experience'
elif factor == 'Factor_2':
    factor_labels[factor] = 'Activity/Engagement'

print("\nFactor Interpretations:")
for factor, label in factor_labels.items():
    print(f" {factor}: {label}")

```

Advanced Analysis

1. Variable Importance Within Groups

```

# Rank variables by loading within each factor
for factor in grouping_table['assignedFactor'].unique():
    group = grouping_table[grouping_table['assignedFactor'] == factor]
    group_sorted = group.sort_values('maxAbsLoading', ascending=False)
    print(f"\n{factor} (by importance):")
    for _, row in group_sorted.iterrows():
        status = "[STRONG]" if row['groupStatus'] == 'STRONG_LOADING' else "
[WEAK]"
        print(f" {row['variable']:20} loading={row['loadingValue']:6.3f}
{status}")

```

2. Calculate Representative Scores

```

# Simple average of standardized variables in each group
import numpy as np
factor_scores = {}
for _, row in grouped_summary.iterrows():
    factor = row['assignedFactor']
    variables = row['variablesInGroup']
    # Create composite score
    factor_scores[factor] = df_prepared[variables].mean(axis=1)

print("Factor Scores Created")

```

3. Feature Selection Based on Groups

```

# Select top loading variable from each group as representative
representatives = []
for _, row in grouped_summary.iterrows():
    variables = row['variablesInGroup']
    # Find highest loading variable
    group_data = grouping_table[grouping_table['variable'].isin(variables)]

```

```
top_var = group_data.loc[group_data['maxAbsLoading'].idxmax(), 'variable']
representatives.append(top_var)

print(f"Representative Variables: {representatives}")
```

Quality Checks for Grouping

1. All Variables Assigned

- Every variable should appear in grouping_table
- Check: `len(grouping_table) == number_of_variables`

2. Reasonable Group Sizes

- Most factors should have multiple variables
- Single-variable factors may indicate weak structure

3. Coherent Interpretation

- Variables in same group make conceptual sense
- If not, review factor loadings

4. Loading Distribution

- Most variables should have STRONG_LOADING status
- High proportion of WEAK_LOADING suggests weak factor structure
- Review data quality and KMO/Bartlett results if concerning

5. Sign Consistency

- Check if negative loadings make sense
- Negative relationships should be interpretable

Threshold Sensitivity

Changing loading_threshold affects the STRONG_LOADING vs WEAK_LOADING classification:

```
Threshold 0.30 → More STRONG_LOADING variables, fewer WEAK_LOADING
Threshold 0.50 → Balanced classification (recommended)
Threshold 0.70 → Fewer STRONG_LOADING, more WEAK_LOADING variables
```

Comparison of Classification:

	threshold=0.30	threshold=0.50	threshold=0.70
STRONG_LOADING	12	8	3
WEAK_LOADING	3	7	12
Total variables	15	15	15

Note: Total variables remain the same; only classification changes. All variables remain in groups.

Common Grouping Patterns

Pattern	Meaning	Implication
Many small groups	Multiple distinct concepts	Rich factor structure
Few large groups	Few underlying themes	Simplified structure
One dominant group	Single primary factor	Strong dimensionality reduction
Balanced groups	Similar importance	Well-balanced concepts
Mixed sizes	Varied concepts	Natural clustering

Post-Grouping Steps

After grouping, typical next steps:

1. Naming Factors

- Use variable names and loadings
- Create intuitive factor labels
- Document meaning

2. Feature Selection

- Choose representative variable(s) per factor
- Use for model input
- Reduces dimensionality

3. Factor Scores

- Calculate score per observation
- Use as new features in models
- Simplifies downstream analysis

4. Documentation

- Record factor definitions
- Save grouping results
- Create interpretation guide

Relationship to Dimensionality Reduction

Factor grouping enables significant dimensionality reduction:

Example:

- Original variables: 50
- Factors extracted: 10
- Strong groups: 8

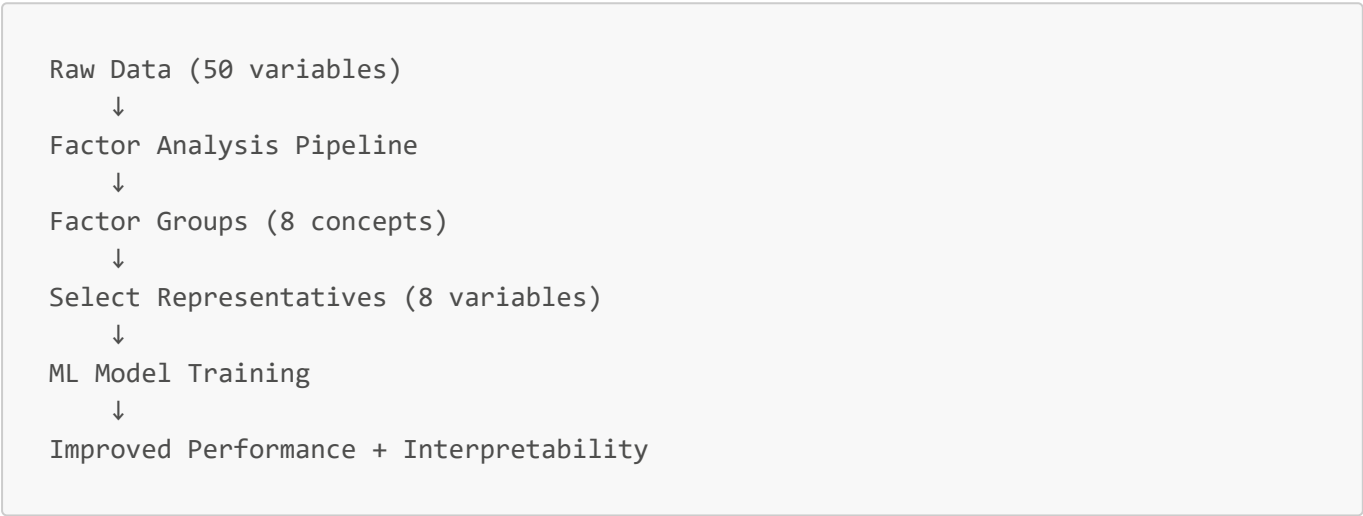
- Representative variables selected: 8

Reduction: 50 → 8 (84% dimensionality reduction)

Benefits:

- Simpler models
- Fewer parameters to estimate
- Reduced overfitting risk
- Faster computation
- More interpretable results

Integration with ML Pipeline



Key Insights

1. **Grouping is interpretation step** - Requires domain understanding
2. **Loading threshold is critical** - Affects group membership
3. **WEAK_LOADING variables are signals** - May indicate data issues
4. **Negative loadings are valid** - Show inverse relationships
5. **Group names matter** - Use intuitive labels for communication
6. **Threshold trade-off** - Lower = more reduction, Higher = more precision

Common Issues and Solutions

Issue	Cause	Solution
Too many WEAK_LOADING	Threshold too high for strict classification	Lower threshold to 0.40; investigate variables
Few group members	Weak factor structure	Review factor loadings and KMO/Bartlett scores
Confusing interpretations	Misaligned variables	Review loading signs; check data quality

Issue	Cause	Solution
Single-variable groups	Factor only matches one variable	May not need that factor; review extraction
Variables span multiple factors	Unclear structure	Review data quality, KMO/Bartlett, consider different thresholds

References

- Thurstone, L. L. (1947). "Multiple-Factor Analysis: A Development and Expansion of The Vectors of Mind"
- Johnson, R. A., & Wichern, D. W. (2007). "Applied Multivariate Statistical Analysis"
- Hair, J. F., Black, W. C., Babin, B. J., & Anderson, R. E. (2014). "Multivariate Data Analysis"

Complete Pipeline Workflow

End-to-End Example

```
from factor_analysis import run_factor_analysis
import pandas as pd

# Load data
df = pd.read_csv("data.csv")

# Run complete pipeline
results = run_factor_analysis(
    df=df,
    target_variable="TARGET",
    drop_last=True,
    fill_strategy_numeric="median",
    encoding_strategy_categorical="ordinal",
    rotation="varimax",
    eigenvalue_threshold=1.0,
    loading_threshold=0.50,
    output_dir="outputs/factor_analysis"
)

# Access results
print("Factors extracted:", results['n_factors'])
print("Variables per factor:")
for _, row in results['grouped_summary'].iterrows():
    print(f"  {row['assignedFactor']}: {row['variablesInGroup']}")

# Save report
report = f"""
FACTOR ANALYSIS REPORT
=====

Step 1: Data Preparation
- Original columns: {len(results['preparation_metadata']['originalColumns'])}
```



```

- Processed columns: {len(results['preparation_metadata']['processedColumns'])}
- Dropped (quality issues): {len(results['preparation_metadata']
['zeroVarianceColumnsDropped'])}

Step 2: KMO Test
- KMO Score: {results['kmo_model']:.4f}
- Interpretation: {results['kmo_metadata']['kmoModel']['interpretation']}

Step 3: Bartlett's Test
- Chi-square: {results['bartlett_chi_square']:.2f}
- P-value: {results['bartlett_p_value']:.8f}
- Conclusion: Reject H0 (factor analysis suitable)

Step 4: Eigenvalues
- Factors with eigenvalue > 1: {results['n_factors']}
- Variance explained: {results['eigen_table']
['cumulativeVarianceRatio'].iloc[results['n_factors']-1]:.1%}

Step 5: Factor Loadings
- Rotation method: {results['factor_loadings_metadata']['rotationMethod']}
- Loadings calculated and rotated

Step 6: Factor Groups
- Factor groups created: {len(results['grouped_summary'])}
- Variables in strong groups: {sum(len(v) for v in results['grouped_summary']
['variablesInGroup'])}
"""

with open("factor_analysis_report.txt", "w") as f:
    f.write(report)

```

Output Files Generated

All outputs saved to `output_dir`:

1. **prepared_data_for_factor_analysis.csv** - Cleaned, encoded, standardized data
2. **kmo_output.json** - KMO test results and interpretations
3. **bartlett_test_output.json** - Bartlett's test results and interpretations
4. **eigenvalues_output.json** - Eigenvalues and variance explained
5. **factor_loadings_output.json** - Factor loadings metadata
6. **factor_grouping_output.json** - Factor grouping logic and definitions
7. **grouping_table.csv** - Detailed variable assignments
8. **grouped_summary.csv** - Factor group summaries

Pipeline Quality Gates

The pipeline includes automatic quality gates:

DATA PREPARATION



```
KMO Test (must pass: KMO > 0.50)
  ↓ PASS
Bartlett Test (must pass: p-value < 0.05)
  ↓ PASS
EIGENVALUE CALCULATION
  ↓
FACTOR LOADINGS
  ↓
FACTOR GROUPING
  ↓
RESULTS & INTERPRETATIONS
```

If KMO or Bartlett fails, pipeline stops with clear error message.

Summary

This factor analysis implementation provides a complete, production-ready pipeline with:

- Automatic data preprocessing and validation
- Dual quality gates (KMO + Bartlett's test)
- Principled factor extraction (Kaiser criterion)
- Improved interpretability (Varimax rotation)
- Actionable grouping and assignments
- Comprehensive logging and metadata
- JSON and CSV output formats
- Full documentation for each step